Experiment 3 – Bellman Equations

NAME:N.AKSHAYA

REG.NO:192425129

COURSE CODE:MLA0305

COURSE NAME:REINFORCEMENT LEARNING

SOLT:B

3.Policyand Value Function Representation using Bellman Equations

Aim:

To implement the Bellman Equation for a simple Markov Decision Process (MDP) and determine the optimal value function and optimal policy using Value Iteration. The experiment also visualizes the state values and optimal actions.

Procedure:

1. Define the states and actions of the environment.
2. Initialize the reward matrix and transition probabilities.
3. Set the discount factor (γ).
4. Initialize all state values to zero.
5. Apply the Bellman Equation iteratively until convergence.
6. Compute the optimal value function.
7. Extract the optimal policy from the value function.
8. Display the summary of results.
9. Visualize the value function and policy using graphs.

Bellman Equation:

$$V(s) = max_a \sum_{s'} P(s'|s,a) [R(s,a,s') + \gamma V(s')]$$

where

- V(s) = Value of state
- γ = Discount factor
- R = Reward
- P = Transition probability

Python code:

```
import numpy as np
import matplotlib.pyplot as plt
# States and Actions
states = ["S0", "S1", "S2", "Goal"]
actions = ["Left", "Right"]
gamma = 0.9
threshold = 0.001
# Transition Model
# (next_state, reward)
transition = {
  0: {
    0: (0, -1),   # Left
    1: (1, 0)     # Right
  },
  1: {
    0: (0, -1),
```

```
        1: (2, 5)
    },
    2: {
        0: (1, 0),
        1: (3, 10)
    },
    3: {
        0: (3, 0),
        1: (3, 0)
    }
}
# Initialize Value Function
V = np.zeros(len(states))
# Value Iteration
iteration = 0
while True:
    delta = 0
    newV = np.copy(V)
    for s in range(len(states)-1):
        values = []
        for a in range(len(actions)):
            next_state, reward = transition[s][a]
            value = reward + gamma * V[next_state]
            values.append(value)
```

```
        newV[s] = max(values)
        delta = max(delta, abs(newV[s]-V[s]))
    V = newV
    iteration += 1
    if delta < threshold:
        break
# Optimal Policy
policy = []
for s in range(len(states)-1):
    values = []
    for a in range(len(actions)):
        next_state, reward = transition[s][a]
        values.append(reward + gamma*V[next_state])
    best_action = actions[np.argmax(values)]
    policy.append(best_action)
policy.append("Goal")
# Summary
print("="*45)
print(" POLICY AND VALUE FUNCTION SUMMARY ")
print("="*45)
print("\nTotal Iterations :", iteration)
print("\nState Values")
for s,v in zip(states,V):
    print(f"{s} : {v:.2f}")
```

```
print("\nOptimal Policy")
for s,p in zip(states,policy):
    print(f"{s} --> {p}")
# Visualization
plt.figure(figsize=(7,5))
plt.bar(states,V,color=['royalblue','orange','green','red'])
plt.title("Optimal Value Function")
plt.xlabel("States")
plt.ylabel("State Value")
plt.grid(axis='y')
plt.show()
plt.figure(figsize=(7,5))
plt.plot(states,V,
         marker='o',
         linewidth=3)
for i in range(len(states)):
    plt.text(i,V[i]+0.3,policy[i],ha='center')
plt.title("Policy and Value Function")
plt.xlabel("States")
plt.ylabel("Value")
plt.grid(True)
plt.show()
```

```
# Policy Visualization
colors=[]
for p in policy:
    if p=="Right":
        colors.append("green")
    elif p=="Left":
        colors.append("orange")
    else:
        colors.append("red")
plt.figure(figsize=(7,4))
plt.bar(states,[1]*len(states),
        color=colors)
for i,p in enumerate(policy):
    plt.text(i,0.5,p,
             ha='center',
             fontsize=12,
             color='white',
             fontweight='bold')
plt.yticks([])
plt.title("Optimal Policy Representation")
plt.show()
```

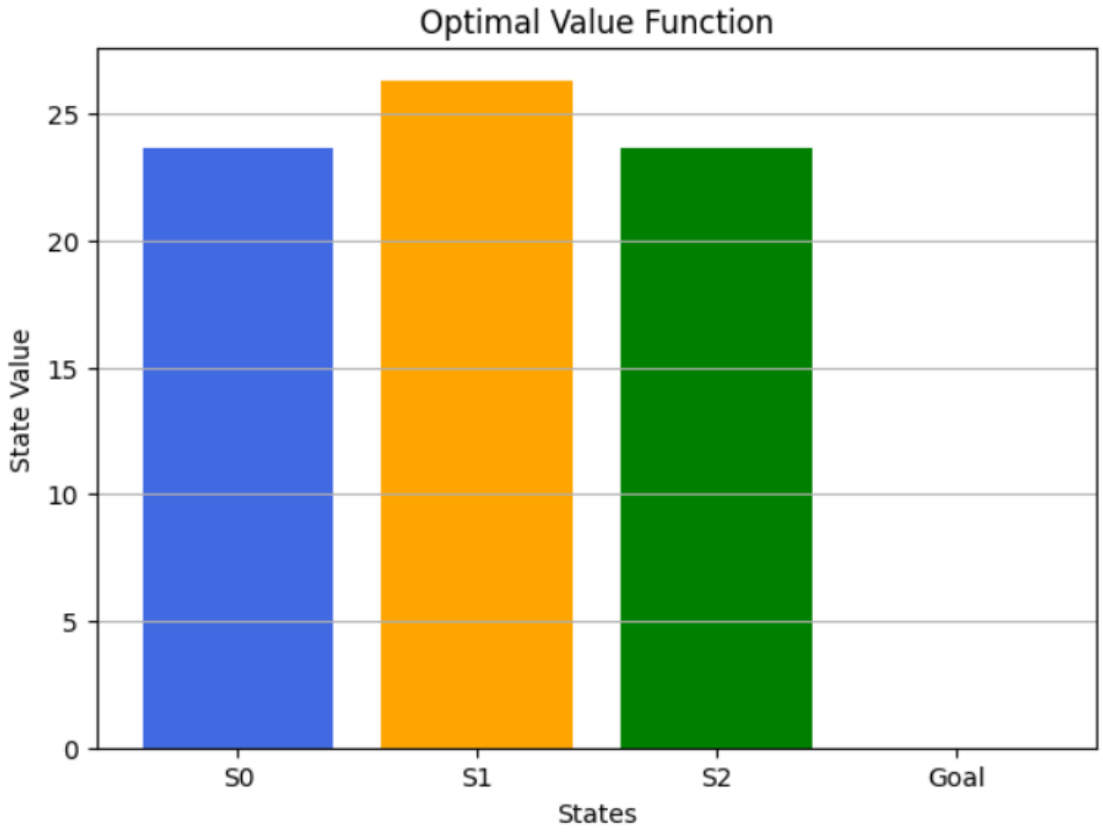
Result:

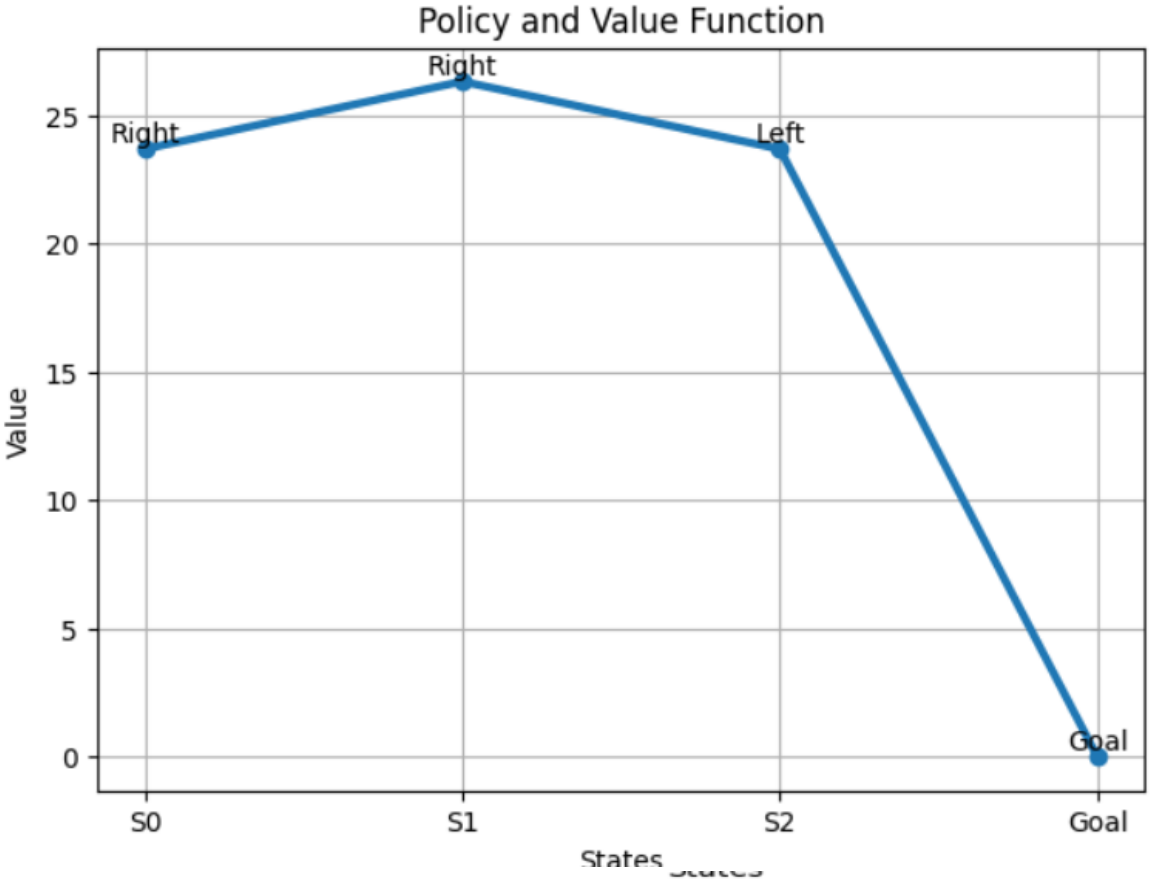
The Bellman Equation was successfully implemented using Value Iteration. The optimal value function and policy were obtained for each state, and the visualizations clearly represented the state values and optimal actions.

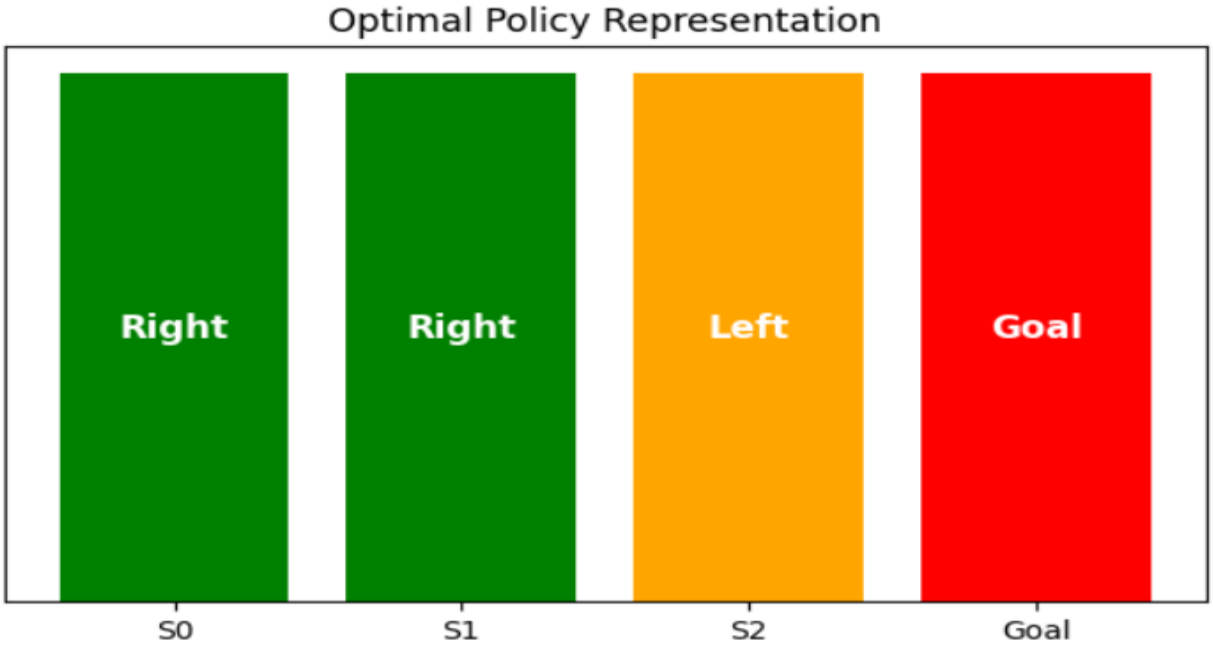
Summary Result:

```
============================================
 POLICY AND VALUE FUNCTION SUMMARY
============================================

Total Iterations : 78

State Values
S0 : 23.68
S1 : 26.31
S2 : 23.68
Goal : 0.00

Optimal Policy
S0 --> Right
S1 --> Right
S2 --> Left
Goal --> Goal
```

Visualization Result




Policy and Value Function
Right
Right
Left
Goal
25
20
15
10
5
0
Value
S0
S1
S2
Goal


Optimal Policy Representation
Right
Right
Left
Goal
S0
S1
S2
Goal